

Sketch-Prompted VLA

Circle-and-Arrow Disambiguation: Scene-Synthesis Method
and Three LIBERO Validation Suites

Aaron

28 July 2026

Abstract

Natural-language instructions for robot manipulation are frequently ambiguous: “pick up the bowl and put it on the plate” does not identify *which* bowl when four identical bowls are on the table. This project proposes resolving it with a **sketch prompt** — a circle drawn around the intended object and an arrow pointing at the intended destination, both drawn directly on the robot’s camera view. This report describes the method I use to synthesise validation scenes for that hypothesis, and the three suites I have built with it on LIBERO-Spatial, LIBERO-Object and LIBERO-Goal: **114 scenes** that are ambiguous *by construction* and machine-scorable. Each scene ships the BDDL task definition that certifies it, so a trained policy can be scored on it automatically. The headline text-only versus text+sketch comparison is not reported here; it requires a trained policy on a GPU. What is reported is the benchmark those rollouts will run against, and the evidence that it measures what it claims to.

1 Problem and approach

A vision-language-action (VLA) policy receives an image and a caption. When the caption underdetermines the task, the policy cannot succeed reliably — not because its manipulation is poor, but because the instruction does not contain the answer. Existing manipulation benchmarks largely avoid this: their scenes usually contain exactly one plausible referent, so caption ambiguity never binds.

My claim is that a **sketch** is a natural and low-bandwidth way for a human to supply the missing information. A circle says *which object*; an arrow says *where it goes*. Both are drawn in image space, so they require no object names, no coordinate frames, and no shared vocabulary between human and robot.

Testing that claim requires scenes where the sketch is not merely helpful but **necessary**. If a scene is solvable from the caption alone, it cannot discriminate between a policy that reads sketches and one that ignores them. The core contribution of this work is a scene-synthesis method that produces such scenes and *proves*, per scene, that they have this property.

2 Scene-synthesis method

The pipeline is deliberately *generative rather than curated*. I author LIBERO task definitions as text, instantiate them in the real simulator, measure the result, and discard anything that fails a gate. Nothing is hand-placed and nothing is accepted on inspection.

2.1 Pipeline

1. **Author the BDDL as text.** LIBERO tasks are declared in BDDL (a PDDL-style language) specifying objects, fixtures, placement regions, an initial state, and a goal predicate. I write this file by direct string templating, injecting duplicate object instances to create the ambiguity and retargeting the (`:goal`) to one specific instance.

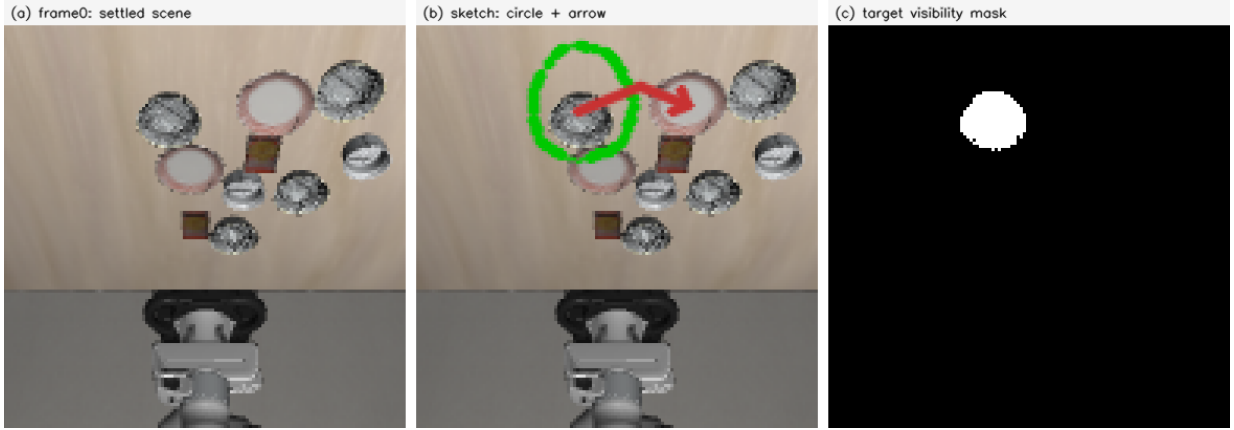


Figure 1: The three artefacts each scene ships (LIBERO-Spatial, `scene_0030`). (a) the settled scene as the robot sees it, 128×128 `agentview`; (b) the same frame with the auto-drawn sketch prompt — a hand-styled circle on the target and an arrow to the destination; (c) the RGB-difference visibility mask used to verify the target is actually visible, not occluded behind another object. Four near-identical bowls are present: the caption alone cannot say which one to move.

2. **Instantiate and settle.** The BDDL is loaded with LIBERO’s `OffScreenRenderEnv` and stepped with zero action until the physics settles, so recorded poses are resting poses rather than spawn poses.
3. **Render and project.** I capture the 128×128 `agentview` frame and project every object’s ground-truth world position into pixel coordinates using the scene’s camera matrix, which is stored per scene.
4. **Draw the sketch.** A circle is drawn around the target’s projected centre and an arrow from target to destination, both with hand-style jitter so the prompt resembles a human annotation rather than a clean overlay.
5. **Gate.** The scene must pass every check in Section 2.3. A failure discards the scene and *restarts* sampling with a fresh seed.
6. **Package.** The scene folder is written with its BDDL, frames, tokens and full provenance.

Every scene records the seed that produced it and is exactly reproducible from it.

2.2 The ambiguity tiers

Ambiguity has two independent axes, and I vary them separately so a failure can be attributed. Each suite contains the same 38-scene tier distribution.

Tier	Scenes	Configuration	What is ambiguous
control	5	1 target, 1 destination	nothing; caption alone suffices
referential	12	N targets, 1 destination	<i>which object</i>
directional	9	1 target, M destinations	<i>which destination</i>
both	12	N targets, M destinations	both axes simultaneously

Table 1: Tier design, identical across all three suites (38 scenes each). The *control* tier is the internal sanity check: a policy that fails there has a manipulation problem, not a disambiguation problem.

2.3 The gate stack

A scene is kept only if it passes all of the following. The two negative oracles are the load-bearing ones — they are what turn “I believe this scene is ambiguous” into “I have verified this scene is ambiguous”.

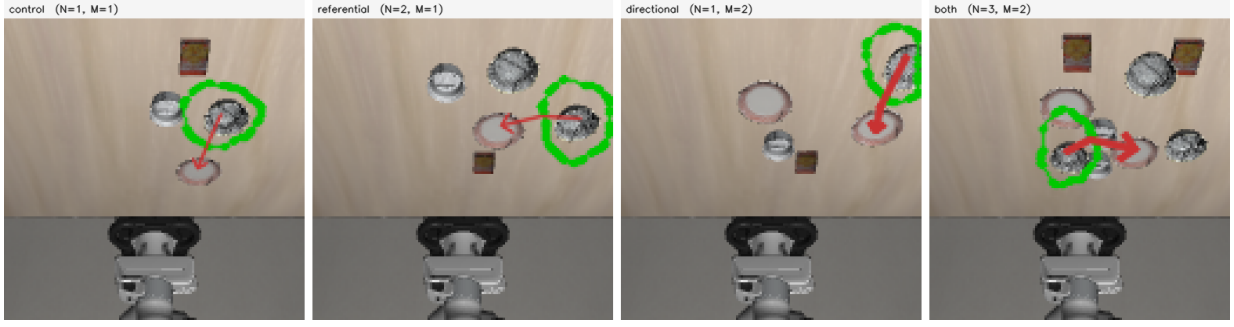


Figure 2: The four tiers, LIBERO-Spatial. N = candidate targets, M = candidate destinations. Left to right the caption becomes progressively less sufficient, while the sketch remains equally informative throughout.

- **Settled** — every body rests above the floor plane; nothing has fallen through or is still in motion.
- **In frame** — target and all candidate destinations project inside the image. A sketch cannot point at something off-screen.
- **Visibility** ≥ 0.35 — the target’s unoccluded fraction, measured by RGB difference against a frame rendered without it. Segmentation masks were tried first and returned empty; the RGB-difference method is what works.
- **Not pre-solved** — `check_success()` is False at $t = 0$, so the goal is not already satisfied by the initial layout.
- **Pixel separation (sketch resolvability)** — candidates must be far enough apart in *image space* for a drawn circle or arrow to distinguish them. Thresholds derive from the projected extents of the objects being told apart, not from a constant: the circle needs $\text{dist}(\text{target}, \text{sibling}) \geq r_{\text{drawn}} + \text{ext}(\text{sibling}) + 3 \text{ px}$. A constant threshold was wrong here, because it scales with the circle while the ambiguity scales with the *sibling*.
- **Positive oracle** — teleporting the target to the named destination makes the BDDL goal evaluate True. This proves the scene is solvable and that the goal I shipped is the goal I meant.
- **Negative oracle, directional** — moving the target to *every other* candidate destination must evaluate False.
- **Negative oracle, referential** — moving *every* same-category sibling to the named destination must evaluate False.
- **Graspable** — a scripted top-down grasp lifts the target more than 3 cm. Enforced in Spatial and Object; recorded but not enforced in Goal, for reasons set out in Section 5.

Why the negative oracles matter. LIBERO’s $\text{On}(a, b)$ predicate is true when a is within 3 cm in xy of b and in contact. Two destination plates placed closer than roughly 6 cm therefore both satisfy the goal — the “wrong” answer scores as correct, and the scene silently stops being a test of anything. The directional negative oracle catches exactly this case, which no amount of visual inspection would have revealed.

Design principle. Where possible I *satisfy a gate by construction rather than by rejection*: sampling is constrained so that valid scenes are produced directly, with rejection as a safety net rather than the primary mechanism. This is why rejection counts (Table 2) are modest despite the number of gates.

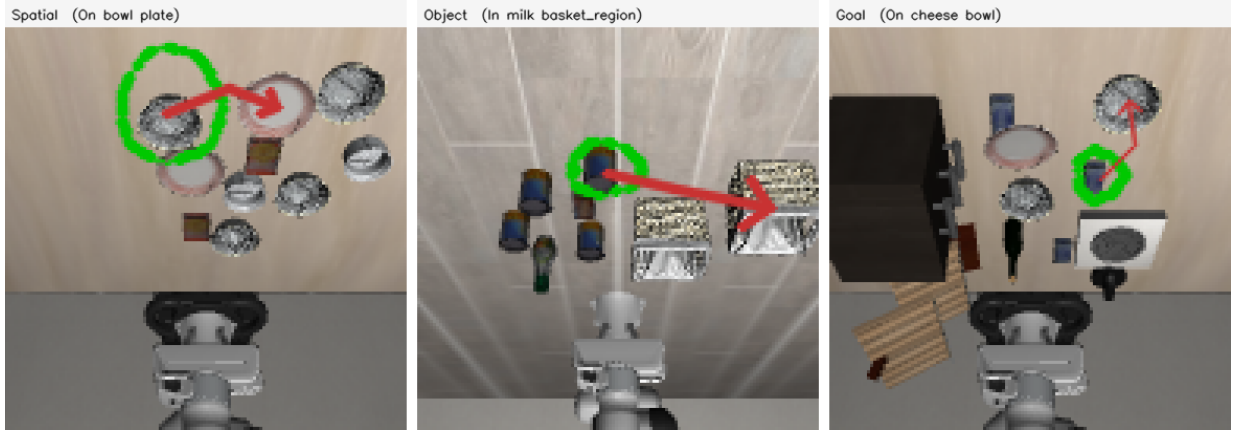


Figure 3: One scene from each suite. The suites differ in workspace, predicate and destination type, but share the tier design, the gate stack and the output schema.

3 The three suites

3.1 Spatial

Built from LIBERO-Spatial’s tabletop pick-and-place. Goal predicate is `(On bowl_t plate_d)`: the destination is an object instance, so `destination_region` equals `destination`. Ambiguity is created by duplicating black bowls (targets) and plates (destinations). Captions are degraded to remove the spatial relation that would otherwise disambiguate — “pick up the black bowl and place it on the plate”.

3.2 Object

Built from LIBERO-Object, where a grocery item is placed *into* a basket. Goal predicate is `(In milk_t basket_d contain_region)`: the destination is a *container region*, so `destination` (the basket) and `destination_region` (its contain region) differ — a distinction the schema records explicitly. Ten grocery categories appear as targets, led by `tomato_sauce` ($\times 6$), `alphabet_soup` ($\times 5$) and `butter` ($\times 5$).

Two facts shaped this suite. Container regions must be *declared* per basket via `(contain_region (:target basket_k))`, or the environment raises `KeyError` on the first step. And object size caps how many identical copies fit the workspace: tall bottles allow at most 3, compact goods 5.

3.3 Goal

Built from LIBERO-Goal, which is structurally different from its siblings: there is no single flat workspace. Each task ships a bespoke scene with real fixtures (`wooden_cabinet`, `flat_stove`, `wine_rack`) and affordance regions. The builder therefore **starts from the original shipped BDDL and injects** duplicates, preserving every fixture, region and `(:init)` line, and retargeting only the goal.

The decisive property is that a Goal destination is either an *object instance* (duplicable, so directional and both tiers are possible) or a *fixed affordance region* (not duplicable). A probe measured the split as 2 object-destination against 6 region-destination tasks. I therefore build all tasks into control and referential tiers, and the 2 object-destination tasks additionally into directional and both. One task — placing a bowl in a drawer — was dropped after all 24 candidate seeds failed the positive oracle: the drawer starts closed, so its region site is retracted and no placement satisfies the goal. Opening then inserting is two actions, which a single circle-and-arrow cannot express. The final roster is 7 tasks, all using `On`.

3.4 Comparison

	Spatial	Object	Goal
Scenes	38	38	38
Goal predicate	On	In	On
Destination type	object instance	container region	object or region
Workspace	table	floor (banded)	per-task, with fixtures
Tasks covered	1	1	7
Visibility	0.963 / 0.999 / 1.025	0.619 / 0.976 / 1.000	0.652 / 0.988 / 1.005
Grasp lift (m)	0.140 / 0.161 / 0.166	0.118 / 0.154 / 0.167	0.000 / 0.142 / 0.167
Clearance xy (m)	0.086 / 0.136 / 0.245	0.110 / 0.160 / 0.326	0.079 / 0.160 / 0.396
Px sep., siblings	24.1 / 34.2 / 49.5	10.6 / 19.5 / 32.4	14.6 / 29.9 / 60.9
Px sep., destinations	22.4 / 35.0 / 49.0	19.1 / 29.4 / 46.9	15.5 / 32.4 / 54.1
Positive oracle	38/38	38/38	38/38
Grasp succeeded	38/38	38/38	32/38
Rejections during build	13	30	17

Table 2: The three suites. Distributions are min / mean / max across the 38 scenes. Pixel separations are reported over the scenes where the corresponding ambiguity exists ($n = 24$ for siblings, $n = 21$ for destinations). The single divergence — Goal’s grasp column — is the subject of Section 5.

4 Output schema and packaging

All three suites conform to one canonical schema (v1.0), so a single loader and a single evaluation harness read every scene. Each scene folder contains its BDDL, the clean frame, the sketch frame, the visibility mask, a model-facing `tokens.json` and a full-provenance `meta.json`.

The canonical fields present in every scene are: `schema_version`, `suite`, `tier`, `target`, `destination`, `destination_region`, `goal_predicate`, `instruction`, `symbolic_tokens` (circle and arrow geometry in pixels), `pick_px`, `place_px`, `radius`, `camera_matrix`, `visibility`, `grasp`, `clearance_xy`, `seed` and `oracle_success`.

Two consumer-facing subtleties. First, `destination_region` is the exact second argument of the BDDL goal and is *not* always the destination instance — for Object it is `basket_2_contain_region` while the destination is `basket_2`. A scorer must compare against `destination_region`; a renderer wanting the physical object should use `destination`. Second, the combined manifest `validation_manifest_all.json` must be keyed on the pair (`suite`, `dir`): each suite numbers its scenes from `scene_0000`, so directory names collide across suites, and seeds overlap as well.

Each suite additionally ships `DATASHEET.md`, a `contact_sheet.png` overview of all 38 scenes, its build log, and both the legacy and canonical manifests.

5 The grasp gate: a deliberate divergence in Goal

Spatial and Object *reject* a scene when the scripted top-down grasp fails. Goal records the result and keeps the scene. Six of its 38 scenes therefore carry `grasp_success: False`. This was a deliberate decision and is worth stating plainly, because it affects how Goal results must be reported.

Why not simply gate it. Table 3 shows the failure is a property of the *object category* under a scripted top-down gripper, not of any particular sampled placement — resampling the seed

Target category	Scenes	Grasp succeeded	Grasp failed
akita_black_bowl	19	19	0
cream_cheese	13	13	0
wine_bottle	4	0	4
plate	2	0	2

Table 3: Goal suite, grasp outcome by target category. The split is perfectly categorical: zero within-category variance.

cannot fix it. Contrast the Object suite, whose 15 `ungraspable` rejections were all graspable grocery categories in unlucky placements, which resampling did fix. The same gate therefore does different work in the two suites: in Object it filters bad samples, whereas in Goal it would filter nothing and instead delete the `wine_on_rack`, `wine_on_cabinet` and `plate_to_stove_front` tasks outright, taking the suite from 7 usable tasks to 4.

What certifies a scene is the positive oracle — teleporting the target to the named destination scores the goal `True` — and that is independent of the scripted grasp. All 38 Goal scenes pass it.

Consequence for scoring. A policy that cannot physically lift a wine bottle will score near zero on those six scenes in *both* the text-only and the text+sketch condition. The comparison remains valid, since both arms face identical physical difficulty, but the floor effect compresses the measured effect size and could mask a real difference. **Results on the Goal suite should therefore be stratified:** the headline number over the 32 grasp-succeeding scenes, with the remaining six reported separately as a manipulation-limited subset. Filter on `meta['grasp']['grasp_success']`; those scenes are flagged `g!` on the contact sheet.

One caveat on the caveat: a lift of 0.02m is a statement about *my scripted* top-down approach, not about the task’s ceiling. A trained policy may well grasp a wine bottle where the scripted oracle cannot, so this should be re-examined once GPU rollouts exist rather than assumed permanent.

6 Verification

Build-time gates are not trusted on their own; the suites are re-verified independently after the fact. `scripts/audit_validation_sets.py` is read-only and re-parses every `scene.bddl` goal predicate directly from disk, checking it against the canonical `target`, `destination_region` and `goal_predicate` recorded in `meta.json` and `tokens.json`. It also checks per-scene file completeness, the full canonical field contract, suite-level packaging, the `(suite, dir)` uniqueness rule, and agreement between the combined manifest and what is actually on disk.

Current status: **clean across all 114 scenes, zero mismatches.**

This audit has already earned its keep. It caught 38 Object scenes whose `meta.json` carried the caption under the legacy key `language` but lacked the canonical `instruction` field — a latent fault that would have raised `KeyError` in any harness following the schema, on a third of the corpus.

7 Limitations

- **Ambiguity is multiplicity, not occlusion.** The `agentview` camera looks down steeply, so objects rarely occlude one another and the visibility gate seldom binds. Scenes are hard because there are many identical candidates, not because the view is cluttered. This corpus should not be cited as evidence about occlusion robustness.

- **Sketches are synthetic.** Circles and arrows are auto-drawn from ground-truth projections with hand-style jitter. Whether humans draw in the same places is an open question and an obvious next experiment.
- **Goal’s harder tiers rest on two tasks.** Only `bowl_on_plate` and `cheese_in_bowl` have duplicable object destinations, so the directional and both tiers derive entirely from them.
- **In is permissive.** Its containment box reaches the floor, so an object merely standing at a basket’s xy position counts as contained. The positive oracle is consequently a weak check in the Object suite; the negative oracles carry the weight.
- **Long-horizon tasks are excluded.** LIBERO-10 requires two or three predicates per goal, which a single circle-and-arrow cannot express. It remains postponed rather than solved.
- **No policy results yet.** Everything reported here concerns the benchmark, not performance on it.

8 Reproduction and next steps

Each builder runs in WSL2 under a conda environment with `robosuite`, `mujoco` and `libero`:

```
python scripts/build_validation_set_spatial_wsl.py
python scripts/build_validation_set_object_wsl.py
python scripts/build_validation_set_goal_wsl.py
python scripts/package_goal_suite.py
python scripts/normalize_validation_schema.py
python scripts/audit_validation_sets.py
```

The last three need no simulator. Note that re-running a builder resamples and overwrites that suite’s 38 scenes.

Next. The immediate blocker is hardware: the text-only versus text+sketch comparison needs a trained policy on a GPU, which the current 4 GB laptop cannot host. Once that exists, the priorities are (i) score all 114 scenes in both conditions, stratifying Goal by grasp; (ii) run a human-agreement study on sketch placement; and (iii) export the corpus to RLDS for UniVLA training, which would require downloading LIBERO-Object and LIBERO-Goal demonstrations, as only Spatial demonstrations were ever fetched.

Contact sheets. Every scene in all three suites is shown on the following page. Tile labels give tier, the $N \times M$ candidate counts, and measured visibility; Goal tiles additionally give the task name and mark grasp failures with `g!`.

A Contact sheets



Figure 4: LIBERO-Spatial, 38 scenes.



Figure 5: LIBERO-Object, 38 scenes.



Figure 6: LIBERO-Goal, 38 scenes. Tiles marked g! are the six scenes where the scripted grasp fails (Section 5).